

# Structures conditionnelles

NSI 1ere – Octobre 2026

exercices sur les structures `if..elif..else` et les boucles non bornées `while`

## 1 Corrigé - Instructions conditionnelles et boucles non bornées

### 1.1 Ex 1 : Vrai ou faux ?

Avec `a = 7`, `b = 12` et `mot = "python"` :

| Expression                             | Valeur              | Explication                                                      |
|----------------------------------------|---------------------|------------------------------------------------------------------|
| <code>a &lt; b</code>                  | <code>True</code>   | <code>7 &lt; 12</code>                                           |
| <code>a == 7.0</code>                  | <code>True</code>   | un entier et un flottant de même valeur sont égaux               |
| <code>a != b and b &gt; 10</code>      | <code>True</code>   | les deux conditions sont vraies                                  |
| <code>not (a &gt; 5)</code>            | <code>False</code>  | <code>a &gt; 5</code> est vrai, donc sa négation est fausse      |
| <code>a &gt; 10 or b &gt; 10</code>    | <code>True</code>   | il suffit qu'une condition soit vraie ( <code>b &gt; 10</code> ) |
| <code>"y" in mot</code>                | <code>True</code>   | la lettre <code>y</code> apparaît dans <code>"python"</code>     |
| <code>b // 5 et b % 5</code>           | <code>2 et 2</code> | <code>12 = 5 × 2 + 2</code>                                      |
| <code>a % 2 == 0</code>                | <code>False</code>  | <code>7 % 2</code> vaut 1 : 7 est impair                         |
| <code>b % 3 == 0 and b % 4 == 0</code> | <code>True</code>   | 12 est divisible par 3 et par 4                                  |

### 1.2 Ex 2 : Blocs et indentation

Script corrigé :

```

1  if chantilly == True:
2      print('gaufre à la chantilly')
3  elif nutella == True:
4      print('gaufre au nutella')
5  else:
6      print('gaufre au sucre')
```

- Si `chantilly = False` et `nutella = False`, aucune condition n'est vraie : le bloc `else` est exécuté, le programme affiche `gaufre au sucre`.
- Si `chantilly = True` et `nutella = True`, le programme affiche seulement `gaufre à la chantilly`. La première condition est vraie : son bloc est exécuté et les conditions suivantes (`elif`) ne sont **pas examinées**.
- Version simplifiée :

```

1  if chantilly:
2      print('gaufre à la chantilly')
3  elif nutella:
```

```

4     print('gaufre au nutella')
5 else:
6     print('gaufre au sucre')

```

### 1.3 Ex 3 : Prédire l'affichage

1. Tableau des affichages :

| valeur de x | affichage du programme A | affichage du programme B |
|-------------|--------------------------|--------------------------|
| 12          | A puis fin               | A puis B puis fin        |
| 7           | A puis fin               | A puis fin               |
| 3           | C puis fin               | fin                      |

Dans le programme B, les deux `if` sont **indépendants** : ils sont tous les deux testés. Dans le programme A, un seul bloc de la structure `if - elif - else` est exécuté.

2. Non, `print("B")` n'est jamais exécuté dans le programme A. Pour atteindre le `elif`, il faut que `x > 5` soit faux, c'est-à-dire `x <= 5`. Mais alors `x > 10` est forcément faux aussi.

3. Il faut tester la condition la plus restrictive en premier :

```

1  if x > 10:
2      print("B")
3  elif x > 5:
4      print("A")
5  else:
6      print("C")
7  print("fin")

```

### 1.4 Ex 4 : input et types

1. Résultats :

- `"3" + "4"` donne `'34'` (concaténation de deux chaînes)
- `int("3") + int("4")` donne `7`
- `float("2.5") * 2` donne `5.0`
- `type(input("Entrer un nombre : "))` donne `<class 'str'>` : `input` renvoie toujours une chaîne

2. Message d'erreur :

```

1  TypeError: '>=' not supported between instances of 'str' and 'int'

```

La variable `age` contient la chaîne `'20'` et non le nombre `20`. Python ne sait pas comparer une chaîne et un entier. Correction :

```

1  age = int(input("Quel âge avez-vous ? "))
2  if age >= 18:
3      print("Vous êtes majeur")
4  else:
5      print("Vous êtes mineur")

```

3. Une taille en mètres est un nombre décimal : on utilise `float`.

```
1 taille = float(input("Entrer votre taille en mètres (ex : 1.75) : "))
```

### 1.5 Ex 5 : Instruction conditionnelle avec cas multiples

1. En testant les seuils du plus grand au plus petit, il suffit d'une borne par condition :

```
1 if t >= 30:
2     print("il va faire très chaud")
3 elif t >= 25:
4     print("il va faire chaud")
5 elif t >= 15:
6     print("il va faire bon")
7 elif t >= 5:
8     print("il va faire froid")
9 else:
10    print("il va faire très froid")
```

2. Résultats attendus :

| t  | affichage              |
|----|------------------------|
| 32 | il va faire très chaud |
| 30 | il va faire très chaud |
| 20 | il va faire bon        |
| 5  | il va faire froid      |
| -3 | il va faire très froid |

Les valeurs **30** et **5** sont des **valeurs limites** (des seuils). Les tester permet de vérifier qu'on a bien choisi **>=** et non **>** : avec **t > 30**, la valeur 30 afficherait à tort « chaud ».

3. Il suffit d'ajouter la saisie avant la structure conditionnelle :

```
1 t = float(input("Entrer la température prévue (en °C) : "))
```

4. Version avec fonction :

```
1 def meteo(t):
2     if t >= 30:
3         return "très chaud"
4     elif t >= 25:
5         return "chaud"
6     elif t >= 15:
7         return "bon"
8     elif t >= 5:
9         return "froid"
10    else:
11        return "très froid"
12
13 # programme principal
14 t = float(input("Entrer la température prévue (en °C) : "))
15 print(f"il va faire {meteo(t)}")
```

## 1.6 Ex 6 : Divisibilité et opérateurs logiques

1. Traduction en Python :

- « `annee` est divisible par 4 » : `annee % 4 == 0`
- « `annee` n'est pas divisible par 100 » : `annee % 100 != 0`
- « `annee` est divisible par 400 » : `annee % 400 == 0`

2. Programme :

```
1 annee = int(input("Entrer une année : "))
2 if (annee % 4 == 0 and annee % 100 != 0) or annee % 400 == 0:
3     print(f"{annee} est bissextile")
4 else:
5     print(f"{annee} n'est pas bissextile")
```

Les parenthèses ne sont pas indispensables (`and` est prioritaire sur `or`), mais elles rendent la condition plus lisible.

3. Vérification :

| année | divisible par 4 et non par 100 | divisible par 400           | bissextile |
|-------|--------------------------------|-----------------------------|------------|
| 2024  | True                           | False                       | oui        |
| 2026  | False (2026 % 4 vaut 2)        | False                       | non        |
| 1900  | False (divisible par 100)      | False (1900 % 400 vaut 300) | non        |
| 2000  | False (divisible par 100)      | True                        | oui        |

## 1.7 Ex 7 : Conditions imbriquées

1. Programme complété :

```
1 age = int(input("Quel est votre âge ? "))
2 etudiant = input("Êtes-vous étudiant ? (oui/non) ")
3 if age < 14:
4     prix = 5
5 else:
6     if etudiant == "oui":
7         prix = 7
8     else:
9         prix = 10
10 print(f"Vous payez {prix} €.")
```

2. Si l'âge n'est pas inférieur à 14 ans (c'est-à-dire `age >= 14`), alors si la personne est étudiante, alors le prix est de 7 €.

3. Version sans imbrication :

```
1 age = int(input("Quel est votre âge ? "))
2 etudiant = input("Êtes-vous étudiant ? (oui/non) ")
3 if age < 14:
4     prix = 5
```

```

5 elif age >= 14 and etudiant == "oui":
6     prix = 7
7 else:
8     prix = 10
9 print(f"Vous payez {prix} €.")

```

*Remarque :* grâce au fonctionnement de `elif`, la condition `age >= 14` est déjà garantie à cette ligne. On pourrait donc écrire simplement `elif etudiant == "oui":`. La version avec `and` a l'avantage de rendre la condition explicite.

### 1.8 Ex 8 : Instruction conditionnelle avec alternative

1. Prix à payer :

- 1 baguette : 1,10 €
- 2 baguettes :  $1,10 + 1,05 = 2,15$  €
- 5 baguettes :  $1,10 + 4 \times 1,05 = 5,30$  €

2. Si `nombre > 1`, la première baguette coûte 1,10 € et les `nombre - 1` suivantes coûtent 1,05 € :

$$prix = 1,10 + (nombre - 1) \times 1,05$$

3. Programme complété (la première ligne doit convertir la saisie en entier) :

```

1 nombre = int(input("Combien de baguettes désirez-vous ? "))
2 if nombre > 1:
3     prix = 1.10 + (nombre - 1) * 1.05
4 else:
5     prix = nombre * 1.10
6 print("Vous avez à payer", round(prix, 2), "euros.")

```

Le bloc `else` traite les cas 0 et 1 baguette (0 baguette donne un prix de 0 €).

4. Sans arrondi, on obtient par exemple `5.3000000000000001` pour 5 baguettes : les nombres à virgule sont stockés de façon approchée par l'ordinateur. Deux solutions pour l'affichage :

```

1 print("Vous avez à payer", round(prix, 2), "euros.")
2 print(f"Vous avez à payer {prix:.2f} euros.")

```

### 1.9 Ex 9 : Boucle `while` et compte à rebours

1. Tableau de suivi :

| compte avant itération | condition <code>compte &gt; 0</code> | affichage                     |
|------------------------|--------------------------------------|-------------------------------|
| 10                     | <code>True</code>                    | 10                            |
| 8                      | <code>True</code>                    | 8                             |
| 6                      | <code>True</code>                    | 6                             |
| 4                      | <code>True</code>                    | 4                             |
| 2                      | <code>True</code>                    | 2                             |
| 0                      | <code>False</code>                   | Décollage ! (après la boucle) |

2. Il y a **5 itérations**. Le variant est `compte` : c'est un entier, positif tant que la boucle tourne, qui diminue de 2 à chaque itération.

3. Avec `compte != 0` :

- en partant de 10, `compte` prend les valeurs 10, 8, 6, 4, 2, puis 0 : la boucle s'arrête, le comportement est identique ;
- en partant de 9, `compte` prend les valeurs 9, 7, 5, 3, 1, -1, -3... et n'est **jamais égal à 0** : c'est une **boucle infinie**.

En revanche, avec `compte > 0` et `compte = 9`, la boucle s'arrête dès que `compte` vaut -1 (après 5 itérations).

4. La condition `compte > 0` est la plus sûre : dès que le variant devient négatif ou nul, la boucle s'arrête, même si on « saute » la valeur 0. Avec `compte != 0`, la boucle ne s'arrête que si `compte` tombe exactement sur 0.

**1.10 Ex 10 : La tirelire**

## 1. Programme modifié :

```

1  epargne = 0
2  semaines = 0
3  while epargne < 150:
4      epargne = epargne + 20
5      semaines = semaines + 1
6  print(f"Il faut {semaines} semaines pour économiser {epargne} €.")

```

## 2. Tableau de suivi :

| <code>semaines</code> avant itération | <code>epargne</code> avant itération | condition <code>epargne &lt; 150</code> |
|---------------------------------------|--------------------------------------|-----------------------------------------|
| 0                                     | 0                                    | True                                    |
| 1                                     | 20                                   | True                                    |
| 2                                     | 40                                   | True                                    |
| 3                                     | 60                                   | True                                    |
| 4                                     | 80                                   | True                                    |
| 5                                     | 100                                  | True                                    |
| 6                                     | 120                                  | True                                    |
| 7                                     | 140                                  | True                                    |
| 8                                     | 160                                  | False → fin de la boucle                |

3. Il faut **8 semaines** ; on a alors économisé **160 €**.

4. Avec `epargne = epargne - 20`, l'épargne prend les valeurs 0, -20, -40... et reste toujours inférieure à 150 : la boucle est **infinie**. La quantité `150 - epargne`, qui devrait être le variant, **augmente** au lieu de diminuer : rien ne garantit plus l'arrêt de la boucle.

**1.11 Ex 11 : Boucle et condition : compter les notes**

## 1. Script complété :

```

1  n = 0
2  au_dessus = 0
3  en_dessous = 0
4  while n < 5:
5      note = float(input("Entrer une note : "))

```

```

6     n = n + 1
7     if note >= 10:
8         au_dessus = au_dessus + 1
9     else:
10        en_dessous = en_dessous + 1
11    print(f"{au_dessus} note(s) >= 10 et {en_dessous} note(s) < 10")

```

2. `n` est un **compteur** : il compte le nombre de notes saisies (le nombre d'itérations) et contrôle l'arrêt de la boucle. `au_dessus` est aussi un **compteur**, mais qui n'augmente que si la condition `note >= 10` est vraie. Il en va de même pour `en_dessous`, qui n'augmente que dans le cas contraire.

3. Tableau de suivi :

| n avant itération | au_dessus avant itération | en_dessous avant itération | condition True/False | note saisie     |
|-------------------|---------------------------|----------------------------|----------------------|-----------------|
| 0                 | 0                         | 0                          | True                 | 12              |
| 1                 | 1                         | 0                          | True                 | 8               |
| 2                 | 1                         | 1                          | True                 | 15              |
| 3                 | 2                         | 1                          | True                 | 9               |
| 4                 | 2                         | 2                          | True                 | 10              |
| 5                 | 3                         | 2                          | False                | (pas de saisie) |

Affichage final : 3 note(s) >= 10 et 2 note(s) < 10

4. À la fin, `au_dessus + en_dessous` vaut toujours 5, c'est-à-dire la valeur de `n`. En effet, à chaque itération, exactement l'un des deux blocs (`if` ou `else`) est exécuté : une seule des deux variables augmente de 1.
5. Oui, le programme se termine toujours. Quelle que soit la note saisie, `n` augmente de 1 à chaque itération. Le variant `5 - n` est un entier positif qui diminue strictement : la boucle s'arrête après exactement 5 itérations.
6. Avec un accumulateur pour la moyenne :

```

1     n = 0
2     au_dessus = 0
3     en_dessous = 0
4     total = 0
5     while n < 5:
6         note = float(input("Entrer une note : "))
7         n = n + 1
8         total = total + note
9         if note >= 10:
10            au_dessus = au_dessus + 1
11        else:
12            en_dessous = en_dessous + 1
13    print(f"{au_dessus} note(s) >= 10 et {en_dessous} note(s) < 10")
14    print(f"Moyenne : {total / n}")

```

Avec les notes de la question 3 : total = 54, moyenne = 10.8.

### 1.12 Ex 12 : Trouver l'erreur

**Programme 1.** La condition `x == 1 or 2` est interprétée comme `(x == 1) or 2`. Python considère le nombre `2` comme vrai (tout nombre non nul est considéré comme vrai), donc la condition est **toujours vraie** : le message s'affiche quel que soit `x`.

```
1 x = int(input("Entrer un nombre : "))
2 if x == 1 or x == 2:
3     print("x vaut 1 ou 2")
```

**Programme 2.** La ligne `i = i + 1` n'est pas indentée : elle est **hors de la boucle**. `i` reste égal à 0, la condition reste vraie et le programme affiche `0` indéfiniment (boucle infinie).

```
1 i = 0
2 while i < 5:
3     print(i)
4     i = i + 1
```

**Programme 3.** La variable `total`, qui intervient dans la condition, n'est **jamais modifiée** dans la boucle : `total < 50` reste vraie, la boucle est infinie et rien ne s'affiche.

```
1 total = 0
2 while total < 50:
3     ajout = 10
4     total = total + ajout
5     print(total)    # affiche 50
```

**Programme 4.** `else` n'est jamais suivi d'une condition : le programme provoque une erreur de syntaxe (`SyntaxError`) dès son lancement.

```
1 note = float(input("Note : "))
2 if note >= 10:
3     print("Admis")
4 else:
5     print("Non admis")
```